



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2022/2023

Sensores e Atuadores
Relatório: Trabalho Lab04

Turma: 11D

Grupo04:

Nome: Rodrigo Nicolau

Número: 50728

Nome: Robert Sbircea

Número: 50732

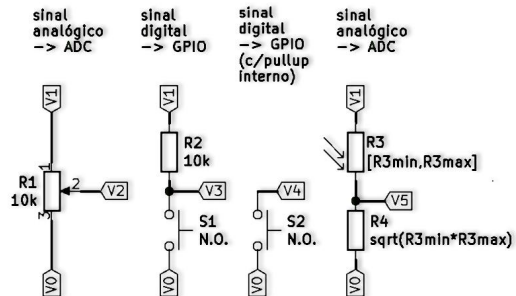
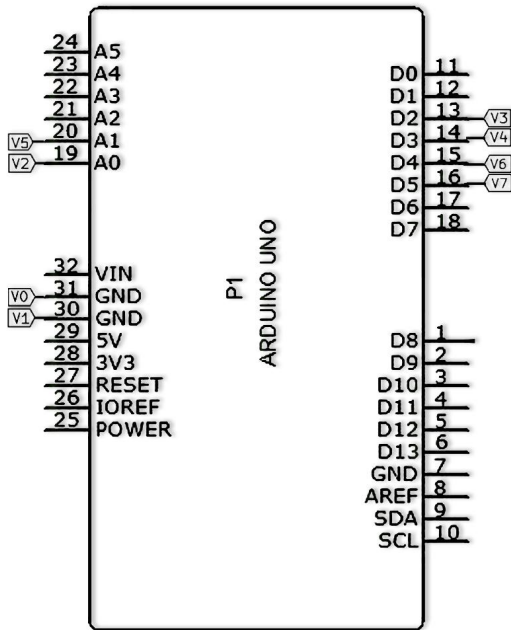
Nome: Fábio Pestana

Número: 50756

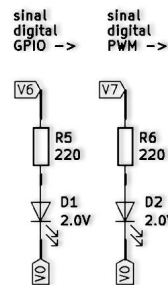
Data: 14 de Novembro 2022

Lab-04

Sensores



Atuadores



Objetivos: Introduzir aos alunos conceitos práticos sobre a programação da placa Arduino, em

C++.

Materiais:

- Breadboard
- Arduino Uno
- Resistências
- Botões
- LDR
- LEDs
- Cabos
- Osciloscópio
- Cabos
- PC

Tarefas:

2 a) Leitura do potenciômetro (V2) - para a leitura deste componente é necessário utilizar o comando `analogRead()` do *pin* A0 (*pin* onde o potenciômetro está ligado ao Arduino), anteriormente definido como PINPOT. Depois é necessário retornar a divisão entre o valor do `analogRead` e 5.0 (voltagem) com 1023, para transformar o valor de bits em tensão.

```
float ler_pinpot()  
{  
  int aux1 = analogRead(PINPOT);  
  return (aux1*5.0)/1023;  
}
```

Calcular a posição do potenciômetro - o cálculo da posição consiste numa função que utiliza uma variável local, sendo que depois retorna a divisão dela com 5.0 volts. Para isso é necessário inserir no void loop uma variável que receba os dados da função `ler_pinpot()`, para depois introduzir esses dados na função `calcularx()`.

```
float calcularx(float T)  
{  
  
  return T/5.0;  
}
```

(código inserido no void loop):

```
V2_analog = ler_pinpot ();  
x = calcularx (V2_analog);
```

Leitura de V3 e V4 - para os interruptores, foi criada uma função booleana que recebe um *pin* (2 ou 3 do Arduino) e utilizando o comando `digitalRead()`, esta função vai ler um valor *High* ou *Low* (1 ou 0), porém como é pedido estes valores ao contrário é necessário retornar a negação de `aux3` (!).

```
bool V3e4(int x)
{
    bool aux3=digitalRead(x);
    return !aux3;
}
```

(código inserido no void loop):

```
V3=V3e4(2);
V4=V3e4(3);
```

Calcular o brilho do LDR - para executar a leitura do brilho é necessário, considerar o valor de $R4$ como o seu valor ótimo neste caso é $\sqrt{100*10}$ logo 10 k Ω , depois utilizando a fórmula do divisor de tensão: $LDR = \frac{V_{DC}*R4}{V5+R4}$ que traduzida para C é: $LDR = (5.00*R4)/(ler_V5()+R4)$.

Para calcular o brilho é necessário utilizar a formula para calcular a resistência do LDR e invertê-la para assim calcular o brilho, ou seja utilizar a seguinte equação: $brilho = \sqrt[{-0.837}]{\frac{LDR}{23.48}}$ esta fórmula traduzida para código fica: `brilho= pow((LDR/23.48), (-1/0.837));`

```
float brilhoLDR()
{
    int R4=10; //valor otimo da resistencia R4
    float LDR = (5.00*R4)/(ler_V5()+R4); //divisor de tensão
    float brilho= pow((LDR/23.48), (-1/0.837)); //apartir da função do brilho
    return brilho;
}
```

Leitura de V5 - tal como o potenciômetro (V2), para a leitura deste componente é necessário o comando `analogRead()` do *pin* A1, anteriormente definido como PINLDR, para assim obteremos o valor da tensão do V5 em bits, através de uma regra de três simples é possível transformar este valor de bits para tensão (entre 0 e 5.00 Volts). Por fim é retornado valor de V5.

```
float ler_V5()
{
    int aux2 = analogRead(PINLDR);
    return (aux2*5.0)/1023;
}
```

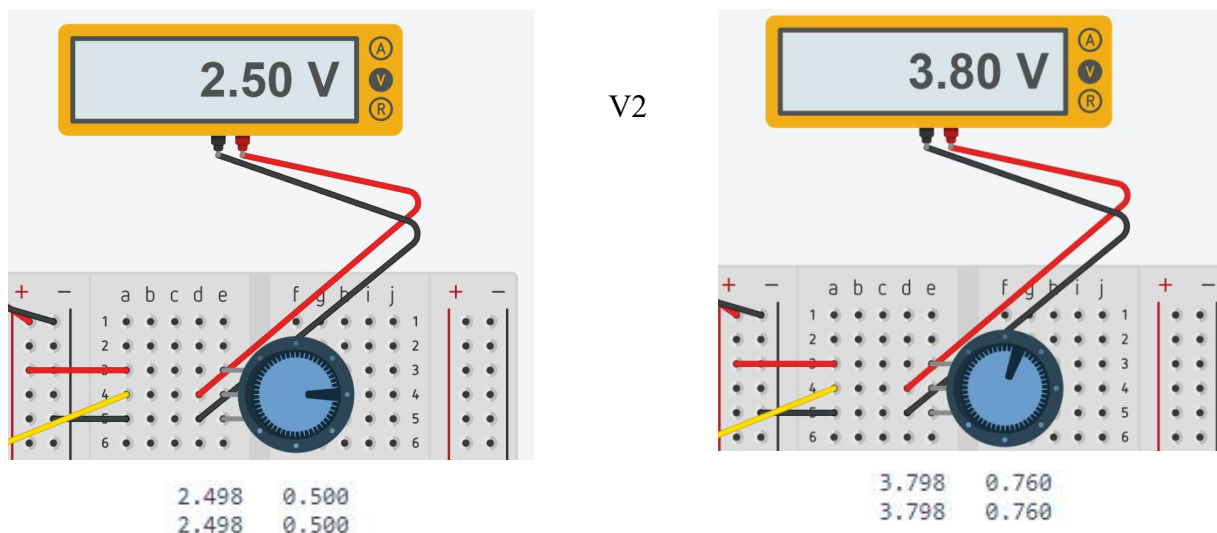
b) Para enviar os valores obtidos em formato CSV para a consola é necessário criar uma função do tipo void, que faça print (`Serial.print()`) dos valores obtidos através das funções desenvolvidas (é necessário criar variáveis globais para assim no void loop chamar as funções e guardar os seus dados nas variáveis). Para desenvolver o formato CSV é necessário fazer `Serial.print("\t")` para assim separar os dados através de um *tab*. Também se fez `Serial.print()` do tempo através de um comando `millis()` sendo que divide-se este valor por 1000.00 para assim obter os segundos em float.

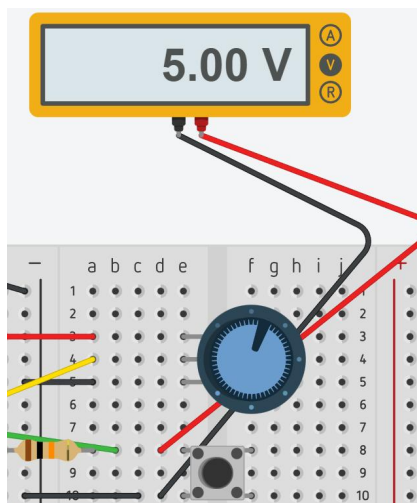
```
void enviarconsola ()
{
    Serial.print(millis()/1000.00); // para o valor ficar em segundos
    Serial.print("\t");
    Serial.print(V2_analog, 3); //algarismos significativos
    Serial.print("\t");
    Serial.print(x, 3);
    Serial.print("\t");
    Serial.print(V3);
    Serial.print("\t");
    Serial.print(V4);
    Serial.print("\t");
    Serial.print(V5_analog, 3); //algarismos significativos
    Serial.print("\t");
    Serial.print(contador);
    Serial.print("\t");
    Serial.print(brilho_Ldr, 3); //algarismos significativos
    Serial.print("\t");
    Serial.print(aux);
    Serial.println("\t");
}
```

c) Para desenvolver uma função que ‘lê’, ‘analisa’ e ‘envia’ dados duas vezes por segundo é necessário criar uma função void que utiliza a função enviarconsola() e em seguida é feito um delay de 500 milissegundos pois é a metade de 1 segundo. Esta função como é chamada no void loop obriga a execução do mesmo duas vezes por segundo.

```
void ciclomeioseg()  
{  
  enviarconsola();  
  delay(500);  
}
```

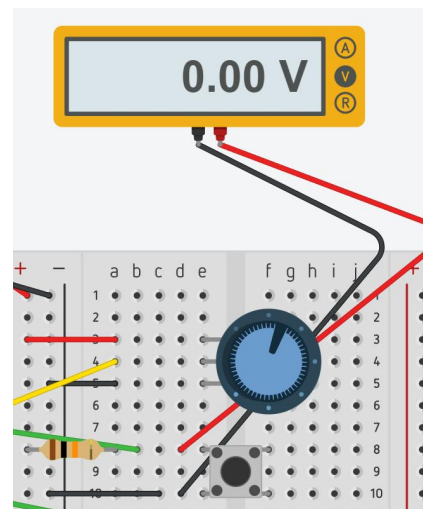
d) Testar hardware com o voltímetro (*tinkercad*):



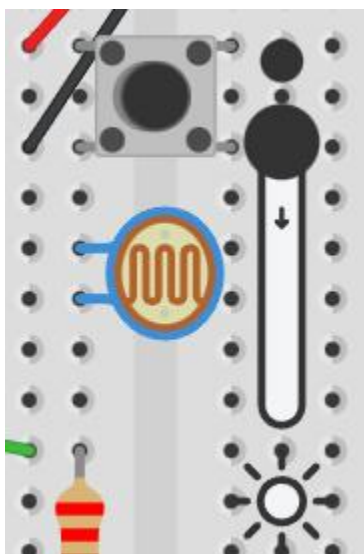
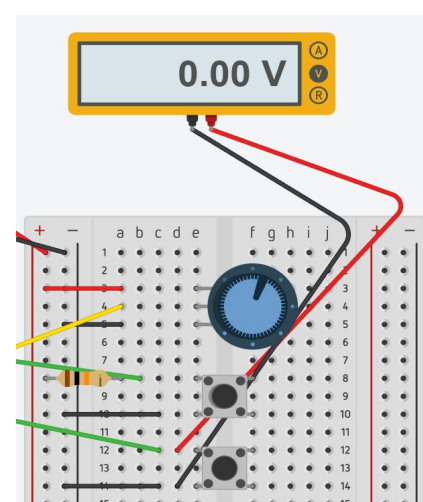
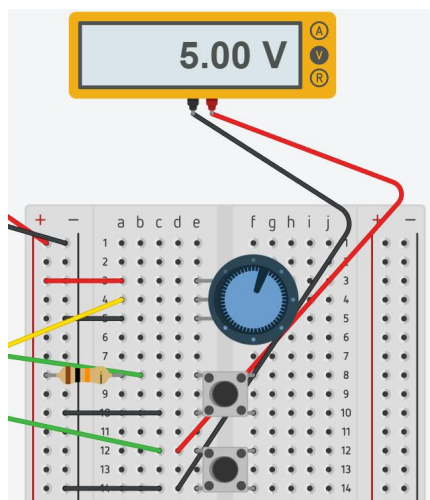


0
0

S1 e S2

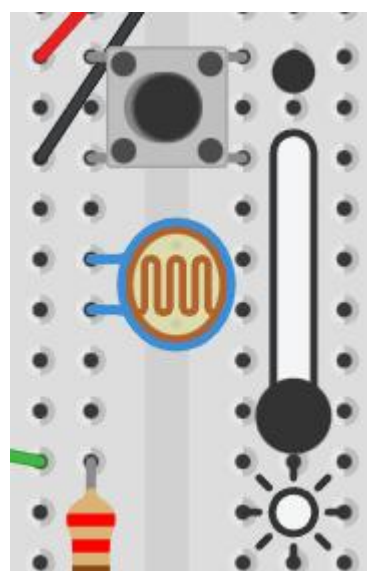


1
1

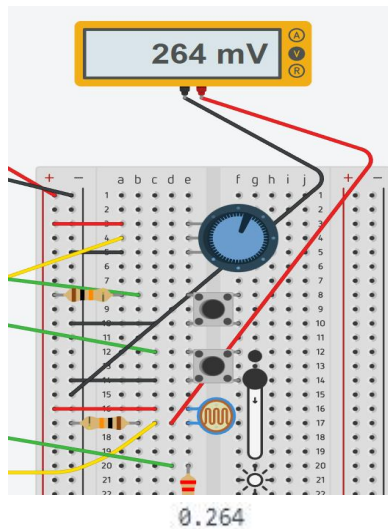


6.547

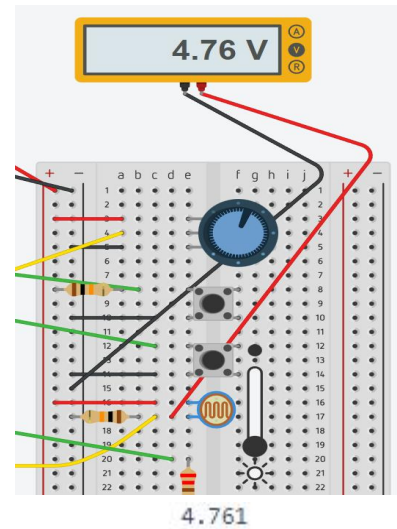
LDR (brilho)
(Mínimo e máximo)



10.106



V5(mínimo e máximo)



3 a) Para piscar o LED D1 desenvolveu-se uma função de tipo void que recebe um float (uma certa frequência entre 0.5 e 50 Hz). Dentro da função utiliza-se o comando digitalWrite() para assim conseguir-se variar o valor do LED entre o seu valor máximo e mínimo (HIGH and LOW).

Como o período T é igual a $1/\text{frequência}$ e o período de $T_1=T_2$ e $T = T_1 + T_2$, logo $T_1=T_2=\frac{1}{2f}$,

sendo que 1segundo é igual a 1000ms conseguimos obter o resultado $500/\text{frequência}$ para o delay pretendido.

```
void pld(float freq)//piscar ledD1
{
    digitalWrite(PIND1, HIGH);
    delay(500/freq);
    digitalWrite(PIND1, LOW);
    delay(500/freq);
}
```

(código inserido no void loop):

```
pld(25); // valor entre 0.5 e 50 Hz
```

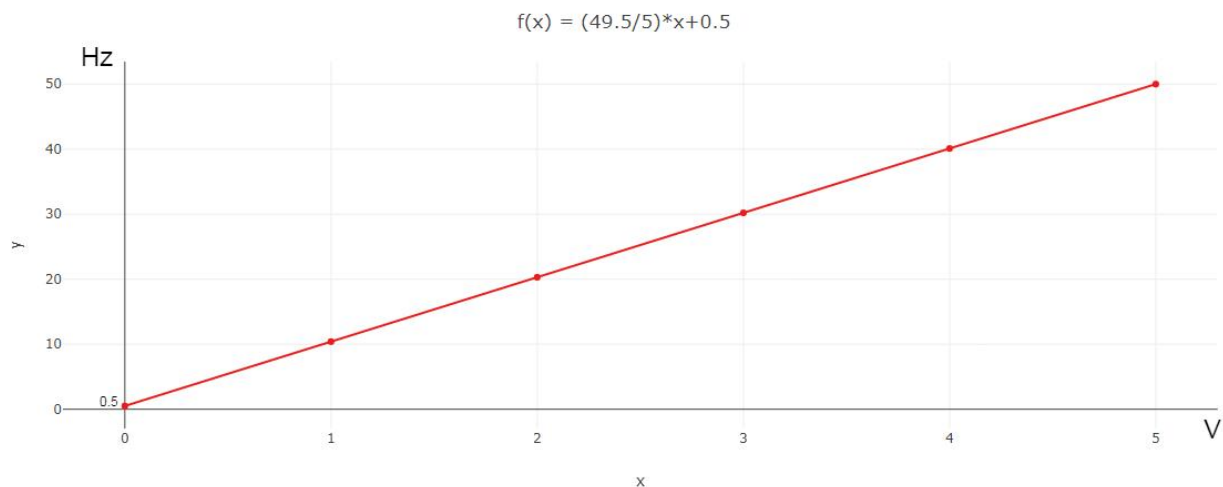
b) Desenvolver código para regular o brilho do LED D2 requer uma função do tipo float que recebe um valor int (percentagem entre 0 e 100), depois cria-se uma variável float para converter a percentagem do brilho num intervalo de 0 a 255. Para gerar uma onda PWM (*pulse width modulation*) utilizou-se o comando analogWrite.

```
float rluz(int brilho)//regulador led D2, brilho entre 0-100%
{
  float aux4=brilho*255/100;      //map(brilho,0,100,0,255)--> outra execução
  analogWrite(PIND2, aux4);
}
```

4 a) Para regular o piscar do LED D1 utilizando o potenciômetro criou-se a seguinte linha de código dentro do void loop:

```
pled((49.5/5)*V2_analog+0.5); // piscar led em função do V2
```

Esta linha de código tem como objetivo utilizar a função pled (função para fazer o LED piscar) e através de uma função afim que interliga o valor da tensão de V2 com a frequência do LED é possível regular o comportamento deste atuador.



b) Desenvolver código que usa um valor enviado na consola para regular o brilho de D2, requer uma função do tipo `int` que recebe um valor inserido na consola e testa se este está entre 0 a 100, ou seja se o seu valor encontra-se em percentagem, em caso de não receber nenhum dado ou se o valor não se encontrar neste intervalo a função vai retornar o valor -1. No void loop é necessário chamar a função e guardar o seu valor numa variável global do tipo `int`, para depois testar se o seu valor é positivo para assim regular o valor do LED D2, através da função anteriormente desenvolvida que regula este atuador.

```
int lervalorserial()
{
  if(Serial.available()>0)
  {
    int aux5=Serial.parseInt();
    if(aux5>=0 && aux5<=100)
    {
      return aux5;
    }
  }
  return -1;
}
```

(código inserido no void loop):

```
aux =lervalorserial();
if (aux>=0)
{
  rluz(aux);
  //delay(100)--> se fosse para regular o LED num intervalo
}
```

c) Para desenvolver código para contar cada segundo que um dos botões foi premido, é necessário criar uma condição dentro do void loop, sendo que esta condição pode ser interpretada da seguinte maneira, se o interruptor S1 ou interruptor S2 encontrarem-se fechados (premidos), ou seja quando retornam valores iguais a 1, uma variável que foi iniciada a 0 (contador) vai adicionando a si própria sempre 1 num intervalo de 1 segundo devido ao delay de 1000 milissegundos.

```
if((V3e4(2)==1) || V3e4(3)==1) //contador do S1 E S2
{
    contador=contador+1;
    delay(1000);
}
```

Conclusão: A atividade laboratorial cumpriu o seu objetivo, pois os alunos conseguiram realizar o código e apresentar um sentido crítico sobre o mesmo.

As maiores dificuldades verificadas nesta atividade foram a realização do código, devido à inexperience dos alunos na interação com o Arduino e no manuseamento do osciloscópio.